Ders 5 Genişletilmiş Notu: Mantıksal Kapılar, Sayı Sistemleri, Tümleyen Gösterimleri ve Kayan Nokta

Ders 5 Genişletilmiş Notu: Mantıksal Kapılar, Sayı Sistemleri, Tümleyen Gösterimleri ve Kayan Nokta

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Önceki Derslerden Köprü
- 2. Veri Saklama: Bit, Byte ve Birimler
- 3. Mantıksal Kapılar ve Boole İşlemleri
- 4. Sayı Sistemleri
- 5. En Anlamlı Bit ve En Düşük Anlamlı Bit
- 6. Taban Dönüşümleri
- 7. Negatif Sayılar: Tümleyen Gösterimleri
- 8. Excess Gösterimi
- 9. İkilik Kesirli Sayılar
- 10. Kayan Nokta Gösterimi

1. Önceki Derslerden Köprü

Hoca derse, önceki ders ve laboratuvarda konuşulan genel konuları hatırlatarak başladı: bilgisayarın tarihçesi, bilgisayarla neler yapılabildiği ve teknolojideki bazı gelişmeler. Bilgisayar artık birçok sorunu çözebiliyor ve insanın yaptığı işleri otomatik olarak yapmaya başlıyor, ama bu ancak bir noktaya kadar geçerli. İnsan beyni hâlâ tam olarak analiz edilebilmiş değil. Araştırmacılar beyindeki sinyalleri izleyerek karar verme mekanizmasını çözmeye çalışıyor. Yapı teorik olarak ortaya konmuş durumda, fakat çalışma prensibinin bazı ayrıntıları bilgisayar üzerinde hâlâ modellenemiyor.

Bu alandaki en son gelişme olarak derin öğrenme (deep learning) yaklaşımını andı. Yapay sinir ağları son dönemde daha yoğun çalıştırılabiliyor. Bunun bir nedeni GPU'lar, RAM ve depolama (storage) gibi donanımların gelişmesi, diğer nedeni de yapay sinir ağlarındaki bazı kısıtların yeni yöntemlerle aşılmasıdır. Bu sayede daha verimli sonuçlar elde edilebiliyor.

Bu dersin asıl konusu ise veri saklamadır (data storage). Bilginin bilgisayarda nerede ve hangi birimlerde saklandığı daha önce konuşulmuştu. Burada bilginin hangi biçimde saklandığına ve bu bilgi üzerinde hangi basit işlemlerin yapıldığına bakılacak.

2. Veri Saklama: Bit, Byte ve Birimler

2.1 Bit ve 0-1 Kullanımının Nedeni

En küçük birim bittir. Bit, *binary digit* ifadesinin kısaltmasıdır ve tek bir basamağın 0 ya da 1 olarak kodlanabildiğini söyler. Bilgisayardaki bütün bilgiler 0 ve 1 kullanılarak kodlanır. Hoca burada şunu sordu: Neden onluk sistem değil de 0 ve 1 kullanılıyor?

Soru: Sebep sinyalin var veya yok olması mı? **Cevap:** Evet. Devre elektriğin olup olmamasıyla beslenir. Örneğin 5 voltluk bir yapıda devreye 5 voltluk gerilim verildiğinde orada pozitif bir

bilgi, yani 1 saklanmış olur. Hiç akım yoksa bu 0 olarak kodlanır.

Soru: Mantıktaki 1 ve 0 olarak da anlaşılabilir mi? **Cevap:** Çok doğru. Bu ikinci kullanım şeklidir ve Boole işlemleri (Boolean operations) buna dayanır. En temel mantıksal işlemler AND, OR, XOR ve NOT'tur. Bu ifadelerle kapılar oluşturulur. Veri saklama, matematiksel işlemler ve karşılaştırma işlemleri bu kapılar üzerinden yapılır.

2.2 Byte ve Adresleme

Bitten sonra gelen birim byte'tır ve 1 byte 8 bittir. Bellekte en küçük adreslenebilen bölge byte'tır, bit değildir.

> **Ders sonrası not:** Altyazıda hoca "bir byte 8 byte'a karşılık geliyordu" diyor (konuşma sürçmesi ya da altyazı hatası olabilir); doğrusu 8 bittir.
>
> **Hocanın vurgusu:** Bellek hücrelerinde adresleme bit bazında değil, byte bazında yapılır. Buna dikkat etmekte fayda var.

2.3 Büyük Birimler

Byte'tan sonra 1024 byte gelir ve bu 1 kilobyte'a karşılık düşer. Bunu 2^{10} byte olarak da yazabiliriz. Sıra megabyte, gigabyte, terabyte, petabyte, exabyte ve zettabyte diye devam eder. Hoca bunların ardından, önceki derste konuşulmayan brontobyte ve son nokta olarak geopbyte adlarını da andı. Üsler 10'ar 10'ar artar:

Birim	Byte cinsinden
1 KB	2^{10}
1 MB	2^{20}
1 GB	2^{30}
1 TB	2^{40}

> **Ders sonrası not:** Hoca zettabyte'tan sonra doğrudan brontobyte'ı saydı; arada büyük olasılıkla yottabyte da vardır. Brontobyte ve geopbyte büyük olasılıkla resmî standart önekler değildir.

2.4 8 Bitle Kaç Değer Yazılır?

8 bit yazıldığında bu bitlerin hepsi 0 da olabilir (`00000000`), hepsi 1 de olabilir (11111111). Hoca 8 bitle kaç farklı sembol ya da değer saklanabileceğini sordu. Cevap $2^8 = 256$'dır. Yani 8 bitte 256 farklı sembol, sayı ya da başka bir bilgi saklanabilir.

Soru: Bu 8 bitle ifade edilebilecek en küçük ve en büyük sayı kaçtır? **Cevap:** 0 ve 255. Ama bu, bit grubunun yalnızca pozitif sayıları ifade ettiği varsayımıyla geçerlidir: `00000000` 0'a, 11111111 255'e karşılık gelir. Bitler negatif sayıları da ifade etmek için kullanılırsa "işin rengi değişir". Bu durum 7. bölümde ele alınacak.

3. Mantıksal Kapılar ve Boole İşlemleri

Boole işlemlerini gerçekleştiren devreler kapılar üzerinden çalışır. Kapılar (logical gates) girdi alır ve çıktı verir. Bazı kapılar iki girdi, bazıları bir girdi alır.

3.1 AND Kapısı

AND kapısı iki girdi (input) alır, bir çıktı (output) verir. İki girdi olduğu için $2^2 = 4$ farklı durum vardır: 00, 01, 10 ve 11. AND kapısı 00, 01 ve 10 için 0, yalnızca 11 için 1 üretir. Mantıktaki karşılığı şöyledir: true ve true yan yana geldiğinde sonuç true olur. Biri false ise sonuç false olur.

3.2 Mantıksal Kapı Tam Olarak Nedir?

Soru: Girdi ve çıktı kısmı tamam, ama mantıksal kapı dediğimiz şey kablolar gibi fiziksel bir şey mi, kod gibi bir şey mi? **Cevap:** En küçük elektronik birim olarak düşünün. Bu birimi girişlerindeki kanallardan elektrikle beslersiniz. AND kapısında iki kanaldan da elektrik geliyorsa çıkış kanalından elektrik üretilir ve bu 1 olarak kullanılır. Birinden gelip diğerinden gelmiyorsa içerideki devre çıkışta elektrik akışı olmamasını sağlar.

Kapıların amacı bilgisayar mühendisliğinde yapılan işleri, yani toplama, çıkarma ve veri saklamayı gerçekleştirmektir. Kapılar bir araya gelerek devreleri, devreler de bu işleri oluşturur. Devrelerin en küçük birimi olarak kapıları düşünebilirsiniz. Bilgisayarlar ve elektronik cihazlar bu kapılar üzerinden tasarlanır, gönderilen sinyaller de bu kapılardan geçer. Elbette başka elektronik birimler de vardır, ama işin başındaki mantık budur. Ayrıntılar Lojik Devreler dersinde görülecektir.

Hoca tarihe de kısaca değindi: Eskiden çok yer kaplayan vakum tüpleri kullanılıyordu. Transistör devreye girdikten sonra bu işlemler hem çok küçük alanlarda yapılmaya başlandı hem de çok hızlandı.

Soru: Mantıksal kapıyı, matematikteki mantığı elektroniğe geçirmek için yapılan parçalar olarak düşünebilir miyiz? **Cevap:** Aynen öyle. Matematikteki mantık işlemleri, 0 ve 1 ya da var ve yok ya da true ve false kavramları üzerinden elektronik devrelere taşınır. Bu kapılar gelişerek bugünkü entegre devreleri oluşturmuştur. Elektronik kartlarda bu kapılar ve benzeri başka kapılar da kullanılır. Yalnızca dört kapı yoktur, bunlar en temel olanlarıdır ve temel işler bunlarla gerçekleştirilir.

3.3 OR Kapısı

OR kapısında da dört farklı giriş vardır. 00 için 0, diğer üç giriş (01, 10, 11) için 1 üretir. Çünkü iki koşuldan birinin sağlanması yeterlidir. Hoca aynı mantığın algoritmalarda da karşımıza çıkacağını söyledi: İki koşulun da sağlanması gerekiyorsa VE, birinin sağlanması yeterliyse VEYA kullanılacak. Sezgi olarak, OR işlemiyle bir nevi toplama, AND işlemiyle de bir nevi çarpma yapıldığı düşünülebilir.

3.4 XOR Kapısı

AND kapısı iki koşulun da sağlandığı durumu, OR kapısı iki koşulun da sağlanmadığı durumu tespit eder. XOR (exclusive or) kapısı ise birinin sağlanıp diğerinin sağlanmadığı durumu kontrol eder. Bu, mantıktaki “ya da” durumudur. Sonuçlar: 00 için 0, 01 için 1, 10 için 1, 11 için 0. Bu ayrım nedeniyle yukarıdaki OR kapısına da inclusive or denir.

Dört durumu tek tabloda görelim:

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

3.5 Bit Düzeyinde İşlem: 0 ile 255

Hocaya göre önemli olan, bitler ya da koşullar işleme girdiğinde bu temel operasyonların nasıl gerçekleştiğini anlamaktır. İki sayının bitleri bitwise işleme girdiğinde karşılıklı gelen bitler eşleşir ve her bit için tablodaki sonuç yazılır. Hoca örnek olarak 0 ile 255’in OR işlemini sordu.

Soru: 0 ile 255’i OR işleminden geçirince 255 tane 1, bir tane 0 mı çıkar? **Cevap:** 255 tane 1 değil, 8 tane 1 var. Burada bitlerin sayısıyla sayının değeri karıştırılmamalıdır.

Bitler karşılıklı yazılınca her 0’ın karşısına bir 1 gelir:

İşlem	Birinci sayı (0)	İkinci sayı (255)	Sonuç
Bitwise OR	`00000000`	`11111111`	`11111111` = 255
Bitwise AND	`00000000`	`11111111`	`00000000` = 0

OR işleminde her bit çifti 0 ve 1 olduğu için her basamağa 1 yazılır, sonuç 255'tir. AND işleminde 0 ile 1'in sonucu 0 olduğundan bütün bitler 0 olur, sonuç 0'dır. Hoca, iki tür işlem olduğunu da ekledi: mantıksal (logic) işlem ve bitwise işlem. İki sayının bitler bazında işleme girmesiyle bütün olarak işleme girmesi arasında fark vardır. Bu fark C programlama dilinde daha sık görülecek ve ileride değerlendirilecek.

3.6 NOT Kapısı

Son kapı NOT'tur ve diğerlerinden farklı olarak bir girdi alır, bir çıktı verir. Girdi 0 ise 1, girdi 1 ise 0 üretir. Adından da anlaşıldığı gibi girdinin değilini (tersini) verir. Hoca bütün işlerin bu dört temel kapı üzerinden kurgulandığını söyledi.

3.7 Kapılarla İlgili Sorular

Soru: 0 ile 255 derken 0 ilk kod, 255 de son kod gibi bir şey mi? **Cevap:** Hoca bunu sayı sistemlerine geçtikten sonra konuşmayı önerdi.

Soru: XOR'un AND ve OR'dan farkı nedir? **Cevap:** XOR, AND ve OR'un ayıramadığı bir farkı yakalamak için kullanılır: 0 ve 1 girdileri verildiğindeki farkı. Aynı girdilerde 0, farklı girdilerde 1 üretir. AND ikisi de doğruyken 1, herhangi biri yanlışken 0 üretir. OR ikisi de yanlışken 0 üretir, birinin doğru olması 1 üretmek için yeterlidir.

Soru: AND ve OR kullanarak XOR elde edilebilir mi? **Cevap:** Hoca bu ayrıntıya şimdilik girmedi. Önce temel mantığın oturması gerektiğini, sorunun yeri gelince tekrar sorulabileceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** XOR, temel kapılarla $A \text{ XOR } B = (A \text{ OR } B) \text{ AND NOT} (A \text{ AND } B)$ biçiminde yazılabilir. Yukarıdaki doğruluk tablosunun dört satırı bu ifadeyle aynen elde edilir.

4. Sayı Sistemleri

Sayı sistemlerini öğrendikten sonra kapılarla belli işler yapılabilecektir. Hoca hangi sayı sistemlerinin olduğunu sordu. Öğrenciler onaltılık, ikilik, onluk ve sekizlik sistemleri saydı. Hoca bunlara, eskiden kullanılan bir sistemi ekledi: Roma rakamları.

4.1 Konumsal Olmayan ve Konumsal Sistemler

İki tip sayı sistemi vardır. Biri pozisyon, yani taban mantığı kullanan konumsal (positional) sistemler, diğeri konumsal olmayan (non-positional) sistemlerdir.

Roma rakamları konumsal olmayan sisteme örnektir. I, V, X, C, M gibi değerler kullanılır ve örneğin 3, 4, 6 ve 9 sayıları III, IV, VI ve IX biçiminde yazılır. Roma rakamlarında basamak mantığı yoktur. Hâlâ bazı yerlerde karşımıza çıksalar da kullanımları kolay olmadığı ve işimizi görmedikleri için tabana dayalı sayılar kullanılır.

Bilgisayar dünyasında en çok kullanılan konumsal sistemler şunlardır: ikilik (binary, taban 2), sekizlik (octal, taban 8), onluk (decimal, taban 10) ve onaltılık (hexadecimal, taban 16). Başka tabanlar da kullanılabilir.

4.2 Onaltılık Sistem

Hoca dört sütunlu bir tablo kurmaya başladı ve önce onluk sütununa 0'dan 16'ya kadar yazdı. Onaltılık sütununda rakam kullanarak 9'a kadar gidilebilir. Tek basamakta "10, 11" diye devam edilemez.

Onaltılık bir sayı birkaç şekilde gösterilebilir. Örneğin 9 için $(9)_{16}$ ya da `0x9` yazılabilir. Onluk için de $(9)_{10}$ yazılır. 9'dan sonra A gelir (`0xA`), sonra B, C, D, E ve F. Hoca öğrencilere nereye kadar gidileceğini sordu ve ipucu verdi: Onluk sistemde bir basamakta en fazla 9'a gidiliyorsa, onaltılık sistemde sayı olarak en fazla 15'e gidilir. Bu yüzden harfler F'de biter:

Onaltılık	A	B	C	D	E	F
Onluk	10	11	12	13	14	15

4.3 16 Sayısı Onaltılıkta Nasıl Yazılır?

Hoca bu soruyu üsteleyerek sordu, çünkü öğrencilerin tahmin ederek düşünmeye alışmasını istiyordu. Verilen yanlış cevaplar kuralı anlamak için yararlıdır:

- `F0` olamaz: F 15'i gösterdiği için bu 15×16 demektir.
- `A0` olamaz: A 10'u gösterdiği için bu 10×16 demektir.
- "1 çarpı F" ya da "0 çarpı F" de değildir: F yalnızca 15'i gösterir.

Doğru cevap `0x10`, yani $(10)_{16}$'dır. Mantığı onluk sistemdeki gibidir: Onlukta 9'dan sonra birler basamağı biter ve "10" yazılır, buradaki 1 onlar basamağını gösterir. Onaltılıkta birler basamağı F'de biter, bir sonraki basamak 16'lar basamağıdır ve "10" yazıldığında değer 16 olur.

Soru: Bunun mantığını anlatabilir misiniz? **Cevap:** Çok basit. Bir sayıyı üç basamaklı düşünün. Onluk sistemde sağdan sola birler, onlar, yüzler basamağı vardır. Onaltılık sistemde birler, 16'lar ve 16'nın karesi yani 256'lar basamağı vardır. Hesap yapılırken basamak değerleri şöyledir:

Sistem	Basamak değerleri (sağdan sola)
Onluk	1, 10, 100
Onaltılık	1, 16, 256
Sekizlik	1, 8, 64
İkilik	1, 2, 4, 8, 16

> **Hocanın vurgusu:** Sayıyı hangi sistemde yazıyorsanız, kapanan parantezin sağ altına mutlaka o sistemin tabanını yazmanız gerekir.

4.4 Sekizlik Sistem

Aradan sonra hoca sekizlik sütununu doldurdu. Rakamlar 0, 1, 2, 3, 4, 5, 6, 7'dir. 7'den sonra basamak dolduğu için 10 yazılır, sonra 11, 12, 13, 14, 15, 16, 17 gelir. Örneğin sekizlikteki 14, onlukta 12'dir ($1 \times 8 + 4$). Sekizlikteki 17, $8 + 7 = 15$ eder. Onluktaki 16 ise sekizlikte 20 olarak yazılır.

4.5 İkilik Sistem ve Tamamlanan Tablo

Hoca onluktaki 16'yı ikilikte yazmak için kaç bit gerektiğini sordu.

Soru: 4 bit yeterli mi? **Cevap:** Hayır. Bir öğrencinin de söylediği gibi, 0'dan başlandığı için 4 bit 15'te biter. 16'yı yazabilmek için 5 bit gerekir.

Tablonun tamamı şöyledir:

Onluk	İkilik	Sekizlik	Onaltılık
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5

Onluk	İkilik	Sekizlik	Onaltılık
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

İkilik yazımların yanına $(1000)_2$ gibi 2 yazılması, sayının ikilik sistemde olduğunu söyler. 0 ve 1'leri görünce ikilik olduğu tahmin edilebilir, ama aynı rakam dizisi başka sistemlerde de geçerlidir. Bu yüzden taban mutlaka yazılır.

> **Ders sonrası not:** Hoca $(1000)_{16}$ değerinin 16 üzeri 4'lük bir sayıya karşılık geldiğini söyledi; doğrusu $16^3 = 4096$'dır (hesapla doğrulandı). Aynı dizi ikilikte 8 eder.

4.6 Genel Kural

16'yı yazmak için bir sonraki basamağa geçilir. İkilik sistemde ilk basamak 2^0'a, sonrakiler $2^1, 2^2, 2^3, 2^4$'e karşılık gelir. Bu, onlukta $10^0, 10^1, 10^2$ ve onaltılıkta $16^0, 16^1$ biçiminde ilerleyen yapının aynısıdır. Hoca bunu genelleştirdi: Basamakları x, y, z, tabanı d olan bir sayıda basamak değerleri sağdan sola $d^0, d^1, d^2, d^3, \ldots$ diye ilerler.

4.7 Baştaki Sıfırlar ve Çizgili Sıfır

Soru: Sayıların başına neden 0'lar yazdık? Değer olarak bir anlamı yok. **Cevap:** Değer değişmez, ama ikilik sayılar genelde 8'in katları kadar bitle ifade edilir. Kaça kadar sayı yazılacaksa o kadar basamak kullanılır, başına da 0 eklenir. Örneğin 16'yı da içeren bir aralık için 5 basamak gerekir. 0-255 aralığı için 8 bit yazılır ve 0 da 8 tane sıfırla ifade edilir. Aksi hâlde işler karışabilir.

Hoca 0'ların içine çizgi çektiğini (Ø) de belirtti. Bu, bölümde 0'ı O harfinden ayırmak için kullanılan ve bazı yerlerde karşılaşılabilecek bir gösterimdir, özellikle ikilik sistemde kullanılır. Öğrencilerin böyle yazması zorunlu değildir.

4.8 0x Gösterimi

Soru: Onaltılık kısımda "0 çarpı 9" gibi yazdık, bunu anlamadım. **Cevap:** Bu bir gösterim şeklidir. Baştaki `0x`, onaltılık sistem kullanıldığını gösterir. `x`, hexadecimal kelimesindeki x'tir, çarpı işareti değildir. Çarpı olarak düşünülürse akıl karışır. Sayıyı başlangıçtan ayırmak için başına 0 eklenmiştir. `0x` ile başlayan her şey onaltılıktır. `0x` kullanılmıyorsa parantezden sonra 16 yazmak gerekir, yani $(9)_{16}$ ya da $(A)_{16}$. İki gösterim de kullanılır.

Soru: Onluktaki 255'i onaltılıkta yazmak için FF mi kullanacağız? **Cevap:** Evet, 255 onaltılıkta `0xFF` olur. Çünkü $16 \times 15 + 15 = 255$.

5. En Anlamlı Bit ve En Düşük Anlamlı Bit

Hangi sistemde çalışılırsa çalışılsın en düşük ve en yüksek anlamlı basamaklar vardır. Hoca bunu 8 bitlik `11100011` sayısı üzerinde gösterdi.

- **En düşük anlamlı bit:** En sağdaki bittir (the least significant bit). Bir başka adı low-order end'dir.
- **En anlamlı bit:** En soldaki bittir (the most significant bit). Bir başka adı high-order end'dir.

Bu adlar neden verilmiştir? Bir öğrenci sayıyı değer olarak en çok yükselten bit olduğunu ve 2^7 kadar değeri bulunduğunu söyledi. Hoca bunu tamamladı: Sağdaki bit değiştirildiğinde sayıda yalnızca 1'lik bir değişiklik olur. Soldaki bit değişirse, örneğin 1 yerine 0 yazılırsa, sayının değeri birden ciddi biçimde değişir. Adlandırma, bu bitlerdeki değişikliğin sayıyı ne kadar etkilediğine göre yapılmıştır.

Ardından hoca bu sayının onluk karşılığını sordu. Basamaklar yazılınca bir kaydırma hatası ortaya çıktı: Bir öğrenci önce 128 + 64 + 32 + 16 + 8 + 4 + 1 dedi. Bu yanlıştı. Soldaki üç bit ile sağdaki iki bit 1'dir, dolayısıyla sonda "4 + 1" değil "2 + 1" vardır:

$$(11100011)_2 = 2^7 + 2^6 + 2^5 + 2^1 + 2^0 = 128 + 64 + 32 + 2 + 1 = (227)_{10}$$

> **Hocanın vurgusu:** Bu, çok sık yapılan hatalardan biridir. Çok hızlı işlem yaparken bu tür kaydırmalar olabilir, bu yüzden adım adım işlem yapın.

6. Taban Dönüşümleri

6.1 227'nin Onaltılık Karşılığı

Hoca aynı sayının onaltılık karşılığını sordu ve bir öğrenci E3 cevabını verdi. Bu, `0xE3` olarak da yazılabilir. Sağlaması şöyledir:

$$(\text{E3})_{16} = 14 \times 16^1 + 3 \times 16^0 = 224 + 3 = (227)_{10}$$

Hoca ayrıca 8 bitin bir byte ettiğini ve bir byte'lık alana yazılabilecek en büyük sayının 8 tane 1 yani 255, en küçük sayının da 0 olduğunu hatırlattı. 255 aynı zamanda `0xFF` olarak yazılır. Parantezler kullanılmayabilir, taban belirtilirken kullanılması tercih edilir.

Biraz sonra bir öğrenci 227'nin onaltılığa nasıl çevrildiğini tekrar sordu. Hoca yöntemi şöyle toparladı:

1. 16'nın kuvvetlerine bakılır. 256, 227'nin içinde yoktur, yani bu sayı üç basamaklı olmaz. Bir alttaki kuvvete, 16'ya inilir.
2. 227, 16'ya bölünür. 22'nin içinde 16 bir kez vardır, 6 kalır. 67'nin içinde 16 dört kez vardır (64), 3 kalır. Bölüm 14'tür.
3. Demek ki 227'nin içinde 14 tane 16 vardır. 14, onaltılıkta E'dir ve 16'lar basamağına E yazılır.
4. Geriye kalan 3, birler basamağına yazılır. Sonuç: $(227)_{10} = (\text{E3})_{16}$.

6.2 33'ün İkilik Karşılığı

Hoca bütün sistemler arasındaki geçişlerin benzer olduğunu söyleyerek 33'ü ikiliğe çevirdi. Bir öğrenci 2'nin kuvvetini önerince hızlı olan yolu gösterdi.

Birinci yol, 2'nin kuvvetleriyle: 33'ün içindeki en büyük kuvvet $2^5 = 32$'dir ve bir tane vardır, 1 kalır. $2^4, 2^3, 2^2$ ve 2^1 yoktur. 2^0 bir tane vardır. Sonuç $(100001)_2$'dir.

İkinci yol, art arda 2'ye bölerek:

Bölünen	Bölüm	Kalan
33	16	1
16	8	0
8	4	0
4	2	0
2	1	0

Son bölüm olan 1 ile kalanlar aşağıdan yukarıya, yani tersten okunur: $(100001)_2$. Hoca, işin 2'ye bölerek de yapılabileceğini ama kuvvetlerle gitmenin daha hızlı olduğunu söyledi.

7. Negatif Sayılar: Tümleyen Gösterimleri

Buraya kadar pozitif tam sayılar konuşuldu. Bilgisayarda negatif sayılar da saklanır ve onlarla işlem yapılır, fakat eksi işareti ayrı bir şekilde gösterilemez. Bu iş de 0 ve 1'ler üzerinden yapılır. Bir öğrencinin söylediği gibi en baştaki bit pozitif ve negatif sayıları ayırmak için kullanılır. Negatif sayıların saklanması için birkaç yöntem vardır: ikiye göre tümleyen gösterimi (two's complement notation) ve excess gösterimi (excess notation). Başka gösterimler de vardır, ama ağırlıklı olarak bunlar kullanılır. Bir gösterim kullanılıyorsa sayılar o gösterimin kurallarına göre yazılır.

7.1 Bire Göre Tümleyen

İkiye göre tümleyene geçmeden önce bire göre tümleyen (one's complement) konuşulmalıdır. Kural basittir: 0'ların yerine 1, 1'lerin yerine 0 yazılır. Örnek sayı `01010001` için sonuç `10101110`'dur.

7.2 İkiye Göre Tümleyen

İkiye göre tümleyen almak için önce bire göre tümleyen alınır, sonra sonuca 1 eklenir:

$$\texttt{01010001} \;\rightarrow\; \texttt{10101110} \;\rightarrow\; \texttt{10101110} + 1 = \texttt{10101111}$$

Hemen ardından eldeli bir durum gösterildi: Sayı `01010000` olsaydı ne olurdu? Bire göre tümleyeni `10101111` olur. 1 eklenince en sağdan başlanarak $1 + 1 = 2$ edilir, 0 yazılır ve 1 elde kalır. Bu elde sol taraftaki 1 ile yine 2 yapar. Sağdaki dört basamağın hepsi böyle 0 olur (dördüncü elde ara adım eklendi). Elde beşinci basamaktaki 0'a eklenince orası 1 olur, kalan basamaklar aynen yazılır:

```
  10101111
+        1
----------
  10110000
```

İkilik sistemde toplama, normal iki sayının eldeli toplamasıdır. Elimizde yalnız 0 ve 1 olduğu için $1 + 1 = 2$ yerine 0 yazılıp 1 elde edilir.

7.3 İşaret Biti ve Negatif Sayının Okunması

Hoca önceki sayıya döndü: 8 tane 1. İşaretsiz sistemde bu 255'tir. Peki işaretli (signed) sistemde, yani negatif sayıların da kullanıldığı sistemde kaçtır?

İkiye göre tümleyen gösteriminde en anlamlı bit işaret biti (sign bit) olarak kullanılır. 1 ise sayı negatif, 0 ise pozitiftir.

Pozitif sayı. `01111111` sayısının en anlamlı biti 0'dır. Pozitif sayılarda hesapta hiçbir şey değişmez, doğrudan hesaplanır ve sonuç 127'dir.

Negatif sayı. `11111111` sayısı tek bitle farklıdır, en anlamlı biti 1'dir. Adımlar şöyledir:

1. Sayı negatif olduğu için başa eksi yazılır.
2. Bire göre tümleyen alınır: `00000000`.
3. 1 eklenir: `00000001`.
4. Çıkan sayının değeri 1'dir, başına eksi konur: -1.

Demek ki 8 tane 1, işaretli sistemde -1'e, işaretsiz sistemde 255'e karşılık gelir.

Soru: İşaret kısmını anladım, ama negatifini hesaplarken neden tümleyen alıyoruz? **Cevap:** Sayının karşılığı matematiksel olarak bir formül doğrultusunda bulunmalıdır, bu bir kuraldır. Neden bire göre değil de ikiye göre tümleyen kullanıldığını hoca az sonra açıkladı (bkz. 7.7).

> **Ek bilgi (derste anlatılmadı):** İkiye göre tümleyenin yaygın kullanılmasının bir başka nedeni büyük olasılıkla toplama devresidir. Bu gösterimde işaretli ve işaretsiz toplama aynı bit işlemiyle yapılabildiği için negatif sayılar için ayrı bir toplama devresi gerekmez.

Soru: 8 tane 1 -1'e indi. 9 tane 1 olsaydı yine -1 mi çıkardı? **Cevap:** 9 tane 1 olsaydı işlem 16 bit üzerinden yapılırdı, -1 çıkmazdı. İşaretli sayılarla çalışırken her zaman 8'in katına tamamlanır.

> **Hocanın vurgusu:** Özellikle işaretli sayılarla çalışırken bit sayısını mutlaka 8'in katlarına tamamlayın.

Soru: Bire göre tümleme matematiksel olarak yalnızca ikiye tümlemeye geçiş için mi var? **Cevap:** Geçiş için kullanılıyor, doğru. Bire göre tümlemenin negatif sayıları ifade etmek için neden kullanılmadığını hoca ayrıca açıkladı (bkz. 7.7).

7.4 10000000 Kaçtır?

Hoca `10000000` sayısının işaretli sistemdeki değerini sordu. Bir öğrencinin -256 cevabı teknik olarak mümkün değildir, çünkü bir bit işaret için ayrılmıştır. Çözüm şöyledir:

1. İşaret biti 1 olduğu için başa eksi yazılır.
2. Bire göre tümleyen alınır: `01111111`.
3. 1 eklenir: `10000000`.
4. Bu bit grubunun değeri, yani 2^7, 128'dir.
5. Sonuç: -128.

7.5 8 Bitlik Aralıklar

8 bitle işaretsiz sistemde 0 ile 255 arası ifade edilir. İşaretli sistemde aralık sorulduğunda "-128 ile 255" cevabı yanlıştır. Doğrusu -128 ile 127 arasıdır.

En büyük pozitif sayı, işaret biti 0 ve geri kalanı 1 olan sayıdır: `01111111` = 127. En küçük sayı `10000000` = -128'dir. Toplam yine 256 farklı değer vardır: 127 pozitif sayı, 0 ve 128 negatif sayı (toplam ara adım eklendi). Bir bit işaret biti olarak ayrıldığı için geriye 7 bit kalır ve bu aralığın üzerine çıkılamaz. Yani işaretli sistemde ifade edilebilen pozitif sayıların sayısı azalır, 128 ya da 255 artık 8 bitle yazılamaz.

Soru: İşaret bitini de işleme katıyor muyuz? **Cevap:** Evet. Sayının eksi mi artı mı olduğuna ondan bakılır, ama hem bire göre hem ikiye göre tümleyen alınırken işaret biti de işleme katılır.

7.6 Bilgisayara İşaretli ya da İşaretsiz Olduğu Nasıl Söylenir?

Soru: Sayı sisteminin işaretli mi işaretsiz mi olduğunu bilgisayara nasıl söylüyoruz? **Cevap:** Örneğin C programlama dilinde işaretsiz bir değişken tipi için başına `unsigned` yazılır. Varsayılan olarak tanımlanan değişken zaten işaretlidir. Her dilde bu ayrım yoktur. Bazı dillerde işaretsiz sistem bulunmaz ve işaretli sistemle çalışmak zorunludur.

> **Ders sonrası not:** C'de varsayılan işaretlilik `int` gibi tipler için geçerlidir; düz `char` tipinin işaretli olup olmaması büyük olasılıkla derleyiciye ve mimariye bağlıdır.

Soru: İşaretli sistemde 255 nasıl yazılır, 16 bit mi gerekir? **Cevap:** Evet. 255'in kendisi 11111111 biçimindedir. Fakat baştaki 1 sayıyı negatif yapar. Başa tek bir 0 eklemek yeterli olurdu ve 9 bit çıkardı, ama 9 bit olarak yazılmaz. 8 ya da 16 gibi 8'in katları kullanılır. Bu yüzden geri kalan bitler 0 ile tamamlanır: `00000000 11111111`. Bu sayı işaretli sistemde 255'e denk gelir.

7.7 Neden Bire Göre Tümleyen Kullanılmıyor?

Bir öğrenci, bire göre tümleyende 0 için iki değer çıktığını ve bir değerin boşa harcandığını söyledi. Hoca bunu doğruladı ve en büyük sorunun bu olduğunu belirtti. `00000000` pozitif sıfırdır. 11111111 yazılıp bire göre tümleyeni alınırsa `00000000` çıkar. Baştaki bit negatif olduğunu gösterdiği için bu desen negatif sıfırı temsil ediyormuş gibi olur. Böylece iki farklı kombinasyonla aynı sayı ifade edilir, yani bir sembol ya da sayı daha az ifade edilebilir.

İkiye göre tümleyende ise 11111111 için 1 eklendiğinde sonuç -1 olur, sıfırın ikinci bir gösterimi kalmaz. Bu yüzden işaretli sayılar için bundan sonra ikiye göre tümleyen kullanılacaktır.

7.8 Negatif Sayıyı Yazmak: −1 Örneği

Şimdiye kadar 0 ve 1'lerden oluşan bir grubun onluk karşılığı bulundu. Tersi nasıl yapılır? Hoca −1'in ikilik yazımını sordu ve en küçük 8'in katı olan 8 bitle yazılması gerektiğini belirtti. 16 bit kullanmak burada doğru olmaz. İki basamakla 11 yazmak da mümkün değildir, her zaman 8'in katları kullanılır.

Bir öğrenci `10000001` yazıp 1 çıkarmayı ve tümleyen almayı önerdi, hoca bunun doğru olmadığını söyledi. "Baştan 8 tane 1 yazmak" da bir yöntem olarak kabul edilmedi. Doğru yöntem şudur:

1. Önce sayının pozitif hâli 8 bitle yazılır: `00000001`.
2. Bire göre tümleyen alınır: `11111110`.
3. 1 eklenir: `11111111`.

Soru: İşaret bitine tümleyen almadan önce bakmıyor muyduk? Buna göre sayı +1 olmuyor mu? **Cevap:** Burada okuma değil yazma yapılıyor. Soru, −1'in nasıl ifade edileceğidir. `00000001` zaten +1'dir ve −1 bunun ikiye göre tümleyeni olarak bulunur.

Sonuç doğru mu? Hoca bunu öğrencilere sordu ve sağlamanın tekrar ikiye göre tümleyen almak olduğunu gösterdi. `11111111` negatiftir, çünkü işaret biti 1'dir. Bire göre tümleyeni `00000000`, 1 eklenince `00000001` yani 1 olur. Başına eksi konur: −1. Demek ki 8 tane 1 ile −1'i ifade etmek doğrudur.

7.9 Kısa Yol: 2^n'den Çıkarmak

Bir öğrencinin önerisiyle hoca kısa yolu gösterdi. İşaret biti 1 olan bir desenin işaretsiz değeri ile işaretli değerinin mutlak değeri toplanınca, o bit sayısının ifade edebildiği sembol sayısına ulaşılır. Tümleyen adı da buradan gelir.

Desen	İşaretsiz değer	$256 - $ işaretsiz değer	İşaretli değer
`11111111`	255	1	−1
`10000000`	128	128	−128

16 bitte toplamın 2^{16}'ya eşit olması gerekir. Hoca bunu 2'nin kuvvetlerini sayarak buldu: $2^{10} = 1024$, $2^{11} = 2048$, $2^{12} = 4096$, $2^{13} = 8192$, $2^{14} = 16384$, $2^{15} = 32768$, $2^{16} = 65536$. Yani 16 bitte işaretsiz değer 65.536'dan çıkarılır ve sonucun başına eksi konur.

> **Hocanın vurgusu:** Bu kolay yol vardır, ama hoca soru çözerken bu yoldan gidilmesini istemediğini söyledi. Tümleyen adımları tek tek uygulanmalıdır.

7.10 32 Bit ve 64 Bit İşlemciler

Soru: Bilgisayarlardaki 32 bit ile 64 bit farkı buradan mı geliyor? **Cevap:** Bu konuyla doğrudan ilgili değil. 32 bit işlemcinin işlem yapabildiği bit sayısı 32'dir. Yani yazılabilecek en büyük sayı 32 tane 1'dir ve bununla işlem yapılır. Daha büyük sayılarla çalışmak için farklı dönüşümler gerekir. RAM tarafında da adreslenebilen alan 2^{32}'ye kadar çıkar. Bu yüzden eski işletim sistemlerinde ve mimarilerde 4 GB'ın ötesi adreslenemez, bunun için farklı yöntemler gerekir. 64 bitte ise iki ayrı 32 bitlik işlemle yapılacak iş, büyük sayılar dahil, tek 64 bitlik işlemle yapılır ve teorik olarak hız iki katına çıkar. Bu önemli bir avantajdır.

8. Excess Gösterimi

Negatif sayıları ifade etmenin bir diğer yolu excess gösterimidir. Kaç bit kullanıldığına göre adı değişir: excess-4, excess-128 gibi.

8.1 Excess-4

Bu gösterim tümleyenden bambaşka kurallara sahiptir. Önce en soldaki biti 1, geri kalanı 0 olan desen belirlenir. 3 bitte bu `100`'dür. İşaretsiz değeri 4 olan bu desen, excess-4'te 0'a

karşılık gelir. Yukarı doğru ilerlendikçe pozitif, aşağı doğru ilerlendikçe negatif sayılar elde edilir:

Bit deseni	Temsil ettiği değer
`111`	3
`110`	2
`101`	1
`100`	0
`011`	-1
`010`	-2
`001`	-3
`000`	-4

Yani excess-4 ile -4 ile 3 arası ifade edilir.

Peki bütün tabloyu yazmadan tek bir desenin değeri nasıl bulunur? Hocanın örneği `011` idi. Bir öğrenci hesabı şöyle yaptı:

$$0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 3, \qquad 3 - 4 = -1$$

Hoca kuralı böyle özetledi: Yazılan bitler kaça karşılık geliyorsa ondan excess değeri çıkarılır.

8.2 Excess-8

4 bitlik gösterimde 0'a karşılık gelen desen `1000`'dir. Hoca birkaç değer sordu: `1111` deseni $15 - 8 = 7$, `0100` deseni ise $4 - 8 = -4$ eder (bir öğrencinin 23 cevabı yanlıştı).

Hoca yer sıkıntısı nedeniyle tablonun üst kısmını tahtaya sıkışık yazdı ve tamamlamayı öğrencilere bıraktı. Tablonun tamamı şöyledir:

Bit deseni	Değer	Bit deseni	Değer
`1111`	7	`0111`	-1
`1110`	6	`0110`	-2
`1101`	5	`0101`	-3
`1100`	4	`0100`	-4
`1011`	3	`0011`	-5
`1010`	2	`0010`	-6
`1001`	1	`0001`	-7
`1000`	0	`0000`	-8

Excess-8 ile 7 ile -8 arası ifade edilir. +8'in karşılığı yoktur, çünkü ifade edilebilen değer sayısı her zaman eksi tarafta bir fazladır.

8.3 Hangi Aralık, Hangi Excess?

Soru: -128 ile 127 aralığı için hangi excess gösterimi gerekir? **Cevap:** Excess-16 değil, excess-128. -4 ile 3 için excess-4, -8 ile 7 için excess-8 kullanıldığı gibi, -128 ile 127 için excess-128 kullanılır.

Soru: Bu gösterimin diğerine göre avantajı ya da dezavantajı var mı? **Cevap:** Hoca bunun IEEE'nin standart bir gösterimi olduğunu ve birçok mimaride negatif sayıların bununla temsil edildiğini söyledi. Performans farkıyla ilgili o an aklına bir şey gelmediğini de belirtti.

> **Ders sonrası not:** Hoca excess gösteriminin IEEE standardı olduğunu ve birçok mimaride negatif sayıların bununla temsil edildiğini söyledi; büyük olasılıkla kastedilen, IEEE 754 kayan nokta standardındaki üs alanıdır. Tam sayıların negatifleri ise günümüz mimarilerinde büyük olasılıkla ikiye tümleyenle saklanır. IEEE 754'te üs kaydırması da büyük olasılıkla tam olarak bu dersteki değerler değildir (32 bitte 128 değil 127).

9. İkilik Kesirli Sayılar

Şimdiye kadar pozitif ve negatif tam sayılarla uğraşıldı. Bilgisayarda saklanmak istenen bir başka tür de kesirli sayılardır. Kesirli bir sayı ifade etmek için akla ilk gelen, nokta koymaktır. Hocanın örneği `1100.110` idi.

Noktanın solunda basamak değerleri sağdan sola $2^0, 2^1, 2^2, 2^3$ diye gider, yani 8 + 4 = 12. Noktanın sağında ise soldan sağa $2^{-1}, 2^{-2}, 2^{-3}$ gelir. 1 olan basamaklar $2^{-1} = \frac{1}{2}$ ve $2^{-2} = \frac{1}{4}$'tür:

$$(1100.110)_2 = 8 + 4 + \frac{1}{2} + \frac{1}{4} = 12\frac{3}{4} = (12{,}75)_{10}$$

Peki sorun nedir? Bir öğrencinin de söylediği gibi, arada nokta vardır ve bunu bilgisayara anlatmak gerekir. Eksi işareti nasıl farklı bir yöntemle anlatıldıysa, nokta da anlatılmalıdır. Bunun için kayan nokta gösterimi kullanılır.

10. Kayan Nokta Gösterimi

10.1 Üç Parça

Kayan nokta gösteriminde (floating point notation) bit grubu, işaret biti ve geri kalanlar diye ayrılmak yerine üçe ayrılır. 8 bitlik örnekte parçalar şöyledir:

Parça	Bit sayısı	Görevi
İşaret biti (sign bit)	1	Sayının pozitif mi negatif mi olduğunu söyler
Üs (exponent)	3	Noktanın kaç basamak ve hangi yöne kayacağını söyler
Mantis (mantissa)	4	Sayının basamaklarını tutar

10.2 Örnek: 0 110 1011

Hoca sayıyı `01101011` olarak verdi. Önce bunun tam sayı olarak karşılığını sordu. Bir öğrencinin 115 cevabı yanlıştı. Basamaklar toplanırken 8 yerine 16 alınmıştı. Doğrusu $64+32+8+2+1 = 107$'dir. Hoca, kendisinin de aynı yanlışa düştüğünü ve kritik noktanın burası olduğunu söyledi: Hesap çok basittir, ama bu kaçırılırsa hatalı iş yapılır. Bilgisayarlarda böyle bir risk yoktur. Onlardaki risk bitin bozulması ya da hatalı aktarılmasıdır. Bilginin doğru aktarılıp aktarılmadığını kontrol eden yöntemler vardır ve hoca bunlara ileride değineceğini söyledi.

Aynı deseni kesirli sayı olarak okumak için bitler parçalanır: `0 | 110 | 1011`.

1. **İşaret:** İlk bit 0 olduğu için sayı pozitiftir.
2. **Mantis:** Mantis alınır ve başına nokta konur: `.1011`.
3. **Üs:** `110` excess-4 ile yorumlanır. 3 bit kullanıldığı için `100` değeri 4'e denk gelir. $6-4 = +2$.
4. **Kaydırma:** Üs +2 olduğundan nokta iki kez sağa kaydırılır: `.1011` $\rightarrow$ `10.11`.
5. **Değer:** Noktanın solu $2^1, 2^0$, sağı $2^{-1}, 2^{-2}$ değerindedir:

$$(10.11)_2 = 2 + \frac{1}{2} + \frac{1}{4} = 2\frac{3}{4} = 2{,}75$$

Demek ki bu kombinasyon kesirli sayı biçiminde saklanıyorsa +2,75'e karşılık gelir. İşaret biti 1 olsaydı $-2{,}75$ olurdu.

Soru: +2'yi nasıl bulduğumuzu tekrar anlatır mısınız? **Cevap:** Üs kısmı excess gösterimine göre tasarlanmıştır. 3 bit olduğu için `100` dörde karşılık gelir, yani excess-4 ile çalışılır. `110` altı eder, altıdan dört çıkar ve iki kalır. Nokta hep mantisin başına konur. Sonuç +2 çıkarsa nokta sağa kaydırılır. -2 çıksaydı sola kaydırılırdı ve `.001011` elde edilirdi. Bu duruma sonraki derste gelinecektir.

Soru: Kayan noktada excess gösterimine göre mi çalışıyoruz? **Cevap:** Evet. Kayan nokta gösteriminde üssün pozitif ya da negatif olması excess gösterimine göre hesaplanır.

Soru: Basamak sayılarını istediğimiz gibi belirleyebiliyor muyuz, yoksa sabit mi? **Cevap:** Bu örnek, 8 bitlik bir alanın kesirli sayı için kullanımını gösterir. Bit sayısı büyüyebilir, 32 ve hatta 64 bit vardır. O zaman üs ve mantis kısımları değişir. İşaret biti değişmez, her zaman tek bittir.

Soru: Bu biçimdeki en büyük sayı 7,5 mi oluyor? **Cevap:** Hoca, zaman nedeniyle bunu sonra birlikte değerlendirmeyi önerdi.

> **Ders sonrası not:** Hoca soruyu cevaplamadı. Bu örnekteki kurallarla en büyük desen büyük olasılıkla `0 | 111 | 1111`'dir: üs $7 - 4 = +3$, `.1111` $\rightarrow$ `111.1` $= 7{,}5$. Yani öğrencinin değeri bu kurallarla tutarlıdır.

10.3 İşaret ile Üssü Karıştırmamak

Hoca dersin sonunda farkı bir kez daha açıkladı. Excess gösterimi, noktanın sağa mı sola mı kayacağına karar vermek için kullanılır. Sayının kendisinin pozitif ya da negatif olması ise işaret bitinde saklanır. İkisi birbirinden farklıdır. Noktanın sağa sola kayabilmesi sayesinde hem daha büyük tam sayılar hem de daha ayrıntılı kesirli sayılar ifade edilebilir. Yapılmasının nedeni budur.

Dersin Toparlaması

Bu derste bilginin bilgisayarda hangi biçimde saklandığı adım adım kuruldu. Her şey 0 ve 1 ile kodlanır, bu değerlerin fiziksel karşılığı elektriğin varlığı, mantıksal karşılığı doğru ve yanlıştır. Bu değerler üzerinde AND, OR, XOR ve NOT kapılarıyla işlem yapılır. Sayılar konumsal sistemlerde tabanın kuvvetleriyle ifade edilir ve ikilik, sekizlik, onluk, onaltılık sistemler arasında geçiş yapılabilir.

Ardından sayıların bilgisayardaki gösterimleri genişletildi. Negatif tam sayılar için ikiye göre tümleyen, üs alanı için excess gösterimi görüldü ve bu ikisi, kesirli sayıları saklayan kayan nokta gösteriminde bir araya geldi. Hoca sonraki derste kesme hatası (truncation error) problemini ve kayan noktayla ilgili birkaç ayrıntıyı konuşacağını, belki bazı örnekler çözeceğini söyledi. Onun ardından muhtemelen algoritmaya giriş yapılacak: algoritma nedir, nasıl tasarlanır ve nasıl ifade edilir.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.